

DAA PROJECT – PART 2

0/1 Knapsack Problem for Logistics Optimization

Design and Analysis of Algorithms | CSC-321 | Spring 2025



Name: Muhammad Anas Khan Farooq

Enrollment: 01-135242-040

Program: BS(IT) 4B

Submitted To: Dr. Saad Mustafa

1. Problem Title

0/1 Knapsack Problem for Logistics Optimization

Source Code:

```
#include <iostream>
#include <stdlib.h>
#include <time.h>

using namespace std;

// ===== GLOBAL DP TABLE (NO STACK OVERFLOW)
// =====
int DP[1001][1001];

// ===== NAPSAK FUNCTION =====

int knapsackDP(int n, int capacity, int weight[], int profit[], bool selected[]) {

    // Initialize first row and column to 0
    for (int i = 0; i <= n; i++) {
        DP[i][0] = 0;
    }
    for (int w = 0; w <= capacity; w++) {
        DP[0][w] = 0;
    }

    // Fill DP table
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= capacity; w++) {
            if (weight[i] <= w) {
                int include = profit[i] + DP[i - 1][w - weight[i]];
                int exclude = DP[i - 1][w];
                if (include > exclude) {
                    DP[i][w] = include;
                }
            }
        }
    }
}
```

```

        }
        else {
            DP[i][w] = exclude;
        }
    }
    else {
        DP[i][w] = DP[i - 1][w];
    }
}
}

```

```

int maxProfit = DP[n][capacity];

```

```

// Backtracking

```

```

int w = capacity;

```

```

for (int i = n; i >= 1; i--) {
    if (weight[i] <= w && DP[i][w] != DP[i - 1][w]) {
        selected[i] = true;
        w = w - weight[i];
    }
    else {
        selected[i] = false;
    }
}

```

```

return maxProfit;

```

```

}

```

```

// ===== MAIN FUNCTION =====

```

```

int main() {

```

```

    // Create arrays

```

```

    int weight[1001];

```

```

    int profit[1001];

```

```

    bool selected[1001];

```

```

int n, capacity;
int choice;

cout << "===== KNAPSACK SOLVER =====" << endl;
cout << "1. Manual Input" << endl;
cout << "2. Random Dataset" << endl;
cout << "Enter choice: ";
cin >> choice;

if (choice == 1) {
    cout << "Enter number of items: ";
    cin >> n;
    cout << "Enter capacity: ";
    cin >> capacity;

    if (n > 1000 || capacity > 1000) {
        cout << "Error: Max 1000 items and 1000 capacity" << endl;
        return 0;
    }

    for (int i = 1; i <= n; i++) {
        cout << "Enter weight for item " << i << ": ";
        cin >> weight[i];
        cout << "Enter profit for item " << i << ": ";
        cin >> profit[i];
    }
}
else if (choice == 2) {
    cout << "Enter number of items: ";
    cin >> n;
    cout << "Enter capacity: ";
    cin >> capacity;

    if (n > 1000 || capacity > 1000) {
        cout << "Error: Max 1000 items and 1000 capacity" << endl;
    }
}

```

```

        return 0;
    }

    int minW, maxW, minP, maxP;
    cout << "Enter min weight: ";
    cin >> minW;
    cout << "Enter max weight: ";
    cin >> maxW;
    cout << "Enter min profit: ";
    cin >> minP;
    cout << "Enter max profit: ";
    cin >> maxP;

    // Generate random data
    srand(time(NULL));
    for (int i = 1; i <= n; i++) {
        weight[i] = minW + (rand() % (maxW - minW + 1));
        profit[i] = minP + (rand() % (maxP - minP + 1));
    }
}

else {
    cout << "Invalid choice" << endl;
    return 0;
}

// Initialize selected array
for (int i = 0; i <= n; i++) {
    selected[i] = false;
}

// Show dataset
cout << endl << "===== DATASET =====" << endl;
for (int i = 1; i <= n; i++) {
    cout << "Item " << i << ": Weight=" << weight[i] << " Profit=" << profit[i] << endl;
}
cout << "Capacity: " << capacity << endl;

```

```

// Time start
time_t startTime = time(NULL);

// Call napsak function
int maxProfit = knapsackDP(n, capacity, weight, profit, selected);

// Time end
time_t endTime = time(NULL);
double executionTime = difftime(endTime, startTime);

if (executionTime == 0) {
    executionTime = 0.001;
}

// Output max profit
cout << endl << "===== RESULT =====" << endl;
cout << "Maximum Profit: " << maxProfit << endl;

// Calculate total weight
int totalWeight = 0;
for (int i = 1; i <= n; i++) {
    if (selected[i] == true) {
        totalWeight = totalWeight + weight[i];
    }
}
cout << "Maximum Weight Used: " << totalWeight << "/" << capacity << endl;

// Show selected items
cout << endl << "Selected Items:" << endl;
bool anySelected = false;
for (int i = 1; i <= n; i++) {
    if (selected[i] == true) {
        cout << "Item " << i << ": Weight=" << weight[i] << " Profit=" << profit[i] << endl;
        anySelected = true;
    }
}

```

```

}
if (!anySelected) {
    cout << "No items selected" << endl;
}

// Performance analysis
cout << endl << "===== PERFORMANCE =====" << endl;
cout << "Execution Time: " << executionTime << " seconds" << endl;

int memoryUsed = (n + 1) * (capacity + 1) * 4;
cout << "Memory Used: " << memoryUsed << " bytes" << endl;

if (memoryUsed > 1024) {
    cout << "Memory Used: " << memoryUsed / 1024 << " KB" << endl;
}

// Complexity analysis
cout << endl << "===== COMPLEXITY =====" << endl;
cout << "Time Complexity:  $O(n * W)$ " << endl;
cout << "Space Complexity:  $O(n * W)$ " << endl;

cout << endl << "===== PROGRAM END =====" << endl;

return 0;
}

```

Output: Manual Input:

```
Microsoft Visual Studio Debug Console
Enter weight for item 1: 5
Enter profit for item 1: 3
Enter weight for item 2: 2
Enter profit for item 2: 4

===== DATASET =====
Item 1: Weight=5 Profit=3
Item 2: Weight=2 Profit=4
Capacity: 4

===== RESULT =====
Maximum Profit: 4
Maximum Weight Used: 2/4

Selected Items:
Item 2: Weight=2 Profit=4

===== PERFORMANCE =====
Execution Time: 0.001 seconds
Memory Used: 60 bytes

===== COMPLEXITY =====
Time Complexity: O(n * W)
Space Complexity: O(n * W)

===== PROGRAM END =====

C:\Users\muham\source\repos\DAA\x64\Debug\DAA.exe (process 32148) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Random Generation:

```
Microsoft Visual Studio Debug Console
===== KNAPSACK SOLVER =====
1. Manual Input
2. Random Dataset
Enter choice: 2
Enter number of items: 3
Enter capacity: 6
Enter min weight: 10
Enter max weight: 89
Enter min profit: 89
Enter max profit: 29

===== DATASET =====
Item 1: Weight=52 Profit=120
Item 2: Weight=53 Profit=111
Item 3: Weight=17 Profit=107
Capacity: 6

===== RESULT =====
Maximum Profit: 0
Maximum Weight Used: 0/6

Selected Items:
No items selected

===== PERFORMANCE =====
Execution Time: 0.001 seconds
Memory Used: 112 bytes

===== COMPLEXITY =====
Time Complexity: O(n * W)
Space Complexity: O(n * W)

===== PROGRAM END =====

C:\Users\muham\source\repos\DAA\x64\Debug\DAA.exe (process 33056) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```